

Python高阶语法

1. 异常的捕获处理

异常（Exception），是我们常说的 bug，可能会出现在：

- 语法错误（Syntax Error）：例如，忘记关闭括号或使用不正确的语法。
- 运行时错误（Runtime Error）：例如，除以零、访问不存在的变量、文件未找到等。

出现异常后，我们可以提前写好异常捕获，出现后对 bug 进行提醒，并继续执行程序。

1.1. 异常捕获的定义语法

定义异常捕获的基本语法如下：

异常捕获定义伪代码 Python

```
1 try:
2     可能发生异常的代码
3 except:
4     发生异常时执行的代码
```

1.2. 捕获指定异常

`except` 后可以指定异常关键字，用来捕获特定的异常（即如果发生我们指定的异常，才捕获）

常见的异常类型如下表所示：

异常类型	描述
<code>EEOFError</code>	当 <code>input()</code> 函数在没有读到数据时引发
<code>ImportError</code>	当导入模块失败时引发
<code>IndexError</code>	当序列索引超出范围时引发
<code>KeyError</code>	试图访问不存在的字典键时引发

<code>KeyboardInterrupt</code>	用户中断执行时（通常是按 Ctrl+C）引发
<code>NameError</code>	尝试访问未声明的变量时引发
<code>SyntaxError</code>	Python 语法错误时引发
<code>ValueError</code>	操作或函数收到具有正确类型但不适当值的参数时引发

▼ 捕获指定异常示例代码

Python |

```

1 try:
2     value = int(input("请输入一个整数: "))
3 except ValueError:
4     print("输入的不是一个有效的整数")
5 except EOFError:
6     print("没有读到任何数据")
7 except Exception: # 这里的Exception代指所有异常
8     print("其他错误")

```

1.3. 捕获所有异常

使用 `Exception` 类可以捕获所有异常：

▼ 捕获所有异常伪代码

Python |

```

1 try:
2     可能发生异常的代码
3 except Exception as e: # e只是个变量名字，也可以是error，自己取名字，用来打印具体的异常
4     print(f"发生异常: {e}")

```

1.4. 捕获异常完整语法

捕获异常完整语法伪代码

Python |

```
1 try:
2     可能发生异常的代码
3 except Exception as e:
4     出现异常时执行的代码
5 else:
6     没有出现异常时执行的代码（一般不用，没慎用）
7 finally:
8     无论是否出现异常都执行的代码
```

捕获异常完整语法示例

Python |

```
1 try:
2     value = int(input("请输入一个整数: "))
3 except ValueError as e:
4     print(f"输入错误: {e}")
5 else:
6     print("输入的是一个有效的整数")
7 finally:
8     print("异常捕获示例结束")
```

1.5. 异常具有传递性

异常具有传递性，即如果在函数内部抛出异常且未处理，则异常会传递到调用该函数的代码中进行处理。

异常的传递性示例

Python |

```
1 def func1():
2     return 1 / 0 # 这里会引发 ZeroDivisionError
3
4 def func2():
5     try:
6         func1()
7     except ZeroDivisionError as e:
8         print(f"在 func2 中捕获到异常: {e}")
9
10 func2() # 输出: 在 func2 中捕获到异常: division by zero
```

2. 类与对象

Python 是一种面向对象的编程语言，类和对象是其核心概念。类是对象的蓝图或模板，对象是类的实例

2.1. 类的定义方法

类的定义语法如下：

类的定义伪代码

Python |

```
1 class 类名:
2     类的属性
3     类的方法
```

其中，类的属性本质上就是变量，但是该变量只属于该类，我们称为**类的属性**

类的方法本质上就是函数，但是该函数只属于该类，我们称为**类的方法**

类的属性和类的方法，统称为**类的成员**

示例代码：

类的定义示例

Python |

```
1 class Person:
2     species = "Homo sapiens" # 类属性
3
4     def greet(self): # 类的方法定义时，第一个参数必须是self，代表类自身，在调用时可
    忽略
5         print(f"Hello, my name is {self.name} and I am {self.age} years old.")
```

2.2. 类的构造方法

构造方法 `__init__` 用于初始化对象的属性，在创建对象时自动调用

```
1 class Person:
2     def __init__(self, name, age):
3         self.name = name
4         self.age = age
5
6 # 创建对象
7 person = Person("Alice", 30)
8 print(person.name) # 输出: Alice
9 print(person.age) # 输出: 30
```

2.3. 类的魔术方法

魔术方法（Magic Methods），也称为双下划线方法（Dunder Methods），是 Python 提供的特殊方法，允许类实现特定的操作。

常见魔术方法如下表所示：

魔术方法	描述
<code>__init__</code>	构造方法，初始化对象
<code>__add__</code>	自定义加法运算
<code>__sub__</code>	自定义减法运算
<code>__mul__</code>	自定义乘法运算
<code>__truediv__</code>	自定义除法运算
<code>__eq__</code>	自定义相等比较运算
<code>__lt__</code>	自定义小于比较运算
<code>__gt__</code>	自定义大于比较运算
<code>__call__</code>	使对象像函数一样可以被调用
<code>__len__</code>	自定义容器类长度
<code>__getitem__</code>	自定义获取容器类中的元素
<code>__setitem__</code>	自定义设置容器类中的元素

```
1 class Vector:
2     def __init__(self, x, y):
3         self.x = x
4         self.y = y
5
6     def __add__(self, other):
7         return Vector(self.x + other.x, self.y + other.y)
8
9     def __eq__(self, other):
10        return self.x == other.x and self.y == other.y
11
12    def __call__(self):
13        print(f"Vector({self.x}, {self.y})")
14
15    def __len__(self):
16        return abs(self.x) + abs(self.y)
17
18    def __getitem__(self, index):
19        return (self.x, self.y)[index]
20
21    def __setitem__(self, index, value):
22        if index == 0:
23            self.x = value
24        elif index == 1:
25            self.y = value
26
27    v1 = Vector(2, 3)
28    v2 = Vector(1, 1)
29    v3 = v1 + v2
30    print(v3.x, v3.y)    # 输出: 3 4
31    print(v1 == v2)     # 输出: False
32    v1()                 # 输出: Vector(2, 3)
33    print(len(v1))       # 输出: 5
34    print(v1[0])         # 输出: 2
35    v1[0] = 10
36    print(v1.x)          # 输出: 10
```

2.4. 类的封装

封装是面向对象编程的基本原则之一，通过将对象的某些属性和方法设为私有，限制外部访问。私有属性和方法前加双下划线

```
1 ▼ class Person:
2 ▼     def __init__(self, name, age):
3         self.__name = name # 私有属性
4         self.__age = age
5
6 ▼     def __display_info(self): # 私有方法
7         print(f"Name: {self.__name}, Age: {self.__age}")
8
9 ▼     def show_info(self):
10         self.__display_info()
11
12 person = Person("Alice", 30)
13 # print(person.__name) # 报错: AttributeError
14 person.show_info() # 输出: Name: Alice, Age: 30
```

2.5. 类的继承与多继承

继承允许一个类继承另一个类的属性和方法，从而实现代码重用。多继承则允许一个类继承多个类。

```

1 class Animal:
2     def __init__(self, name):
3         self.name = name
4
5     def speak(self):
6         pass
7
8 class Dog(Animal):
9     def speak(self):
10         return "Woof!"
11
12 class Cat(Animal):
13     def speak(self):
14         return "Meow!"
15
16 class Robot:
17     def operate(self):
18         return "I am a robot."
19
20 class RoboDog(Dog, Robot):
21     def __init__(self, name):
22         Dog.__init__(self, name)
23         Robot.__init__(self)
24
25 robo_dog = RoboDog("Rex")
26 print(robo_dog.name)      # 输出: Rex
27 print(robo_dog.speak())  # 输出: Woof!
28 print(robo_dog.operate()) # 输出: I am a robot.

```

注意，类的私有成员并不会被继承，但是可以通过类的公开方法访问私有成员

3. 模块与库

模块和库是Python中用于组织和重用代码的方式。模块是包含函数、类和变量的文件，而库则是提供特定功能的模块集合

3.1. 模块的定义与使用

模块就是一个包含Python定义和语句的文件（.py文件）。通过导入模块，可以在其他文件中使用其定义的函数、类和变量。

创建一个模块非常简单，只需要编写一个Python文件并定义一些函数或变量即可。例如，创建一个名为 `mymodule.py` 的文件：

▼ 模块示例

Python |

```
1 # mymodule.py
2
3 def greet(name):
4     print(f"Hello, {name}!")
5
6 PI = 3.14159
```

使用模块

可以使用 `import` 关键字导入模块并访问其中的函数和变量：

▼ 模块导入示例1

Python |

```
1 # main.py
2
3 import mymodule
4
5 mymodule.greet("Alice") # 输出: Hello, Alice!
6 print(mymodule.PI)      # 输出: 3.14159
```

可以使用 `from ... import ...` 语法从模块中导入特定的函数或变量：

▼ 模块导入示例2

Python |

```
1 # main.py
2
3 from mymodule import greet, PI
4
5 greet("Bob")           # 输出: Hello, Bob!
6 print(PI)               # 输出: 3.14159
```

还可以为导入的模块或函数设置别名：

▼ 模块导入示例3

Python |

```
1 # main.py
2
3 import mymodule as mm
4
5 mm.greet("Charlie")    # 输出: Hello, Charlie!
6 print(mm.PI)           # 输出: 3.14159
```

3.2. 库

创建一个库与创建模块类似，只是库通常由多个模块组成，并且需要一个特殊的文件 `__init__.py`

`__init__.py` 这个文件可以是空的，但它的存在表示当前目录（文件夹）是一个Python包
一个自定义库的目录结构如下

```
1 my_library/
2     __init__.py
3     module1.py
4     module2.py
```

▼ module1.py文件内容

Python |

```
1 # module1.py
2 def function1():
3     print("This is function1 from module1")
```

▼ module2.py文件内容

Python |

```
1 # module2.py
2 def function2():
3     print("This is function2 from module2")
```

`__init__.py` 文件可以为空，也可以是如下内容，使得库的导入和管理更加方便

▼ __init__.py文件内容

Python |

```
1  # __init__.py
2  from .module1 import function1
3  from .module2 import function2
4
5  # __all__ = [module1] # 可以通过指定__all__列表来限制公开的模块
```

如此，就可以如下方式使用该库

▼ main.py引入上述自定义库

Python |

```
1  # main.py
2  import my_library
3
4  my_library.function1() # 输出: This is function1 from module1
5  my_library.function2() # 输出: This is function2 from module2
```

3.3. 标准库

Python 标准库（Standard Library）是Python随附的一组模块，提供了广泛的功能，包括文件 I/O、系统调用、网络通信、文本处理等。以下是一些常用的标准库模块：

模块	描述
<code>os</code>	提供与操作系统交互的功能
<code>sys</code>	提供与Python解释器相关的功能
<code>math</code>	提供数学函数
<code>datetime</code>	提供日期和时间处理功能
<code>json</code>	提供JSON数据的解析和生成功能
<code>re</code>	提供正则表达式匹配操作
<code>random</code>	提供生成随机数的功能
<code>collections</code>	提供高级数据结构

示例代码：

python常见标准库示例

Python |

```
1  import os
2  import sys
3  import math
4  import datetime
5  import json
6  import re
7  import random
8  import collections
9
10 # 使用 os 模块
11 print(os.getcwd()) # 输出当前工作目录
12
13 # 使用 sys 模块
14 print(sys.version) # 输出 Python 解释器的版本
15
16 # 使用 math 模块
17 print(math.sqrt(16)) # 输出: 4.0
18
19 # 使用 datetime 模块
20 now = datetime.datetime.now()
21 print(now) # 输出当前日期和时间
22
23 # 使用 json 模块
24 data = '{"name": "Alice", "age": 25}'
25 parsed_data = json.loads(data)
26 print(parsed_data) # 输出: {'name': 'Alice', 'age': 25}
27
28 # 使用 re 模块
29 pattern = re.compile(r'\d+')
30 match = pattern.search('The answer is 42')
31 print(match.group()) # 输出: 42
32
33 # 使用 random 模块
34 print(random.randint(1, 10)) # 输出: 1 到 10 之间的随机整数
35
36 # 使用 collections 模块
37 counter = collections.Counter(['a', 'b', 'c', 'a', 'b', 'a'])
38 print(counter) # 输出: Counter({'a': 3, 'b': 2, 'c': 1})
```

3.4. 第三方库

第三方库是由Python社区开发并发布的库，通常需要通过 `pip` 工具进行安装。以下是一些常用的第三方库：

注意，这仅仅是冰山一角，可以自行探索

库	描述
<code>requests</code>	简单高效的HTTP请求处理
<code>numpy</code>	提供高效的数组和矩阵运算
<code>pandas</code>	提供数据分析和数据操作功能
<code>matplotlib</code>	提供绘图和数据可视化功能
<code>scipy</code>	提供科学计算功能
<code>flask</code>	用于构建Web应用的微框架
<code>beautifulsoup4</code>	解析和处理HTML和XML文件

安装和使用第三方库

可以使用 `pip` 工具安装第三方库。例如，安装 `requests` 库，可以在命令行中输入

```
1 pip install requests
```

安装完成后，可以在代码中导入并使用该库：

▼ requests库使用示例 Python |

```
1 import requests
2
3 response = requests.get('https://api.github.com')
4 print(response.status_code) # 输出: 200
5 print(response.json())      # 输出: GitHub API 的响应数据
```